

Contents

Classification of Floating-Point Numbers	1
Introduction	1
Floating point numbers (16 bit)	1
Types of floating-point number	1
Classification criteria	1
Infinity	1
QNaN	1
SNaN	2
Zero	2
Subnormal	2
Normal	2
Verilog Code	3
Verification	4
Testbench	4
Output	6
Conclusion	6
Multiplication of floating point	7
Pseudo code	Error! Bookmark not defined.
Verilog code	Error! Bookmark not defined.

Classification of Floating-Point Numbers

Introduction

Floating point numbers (16 bit)

The Verilog code is written to take 32, 64 and 128 bit values but for simplicity's sake I shall consider 16 bit floating point representation

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Sign	Exponent					Mantissa									

Types of floating-point number

1. Signalling Not a Number (SNaN)
2. Quiet Not a Number (QNaN)
3. Infinity ($+\infty$, $-\infty$)
4. Zero (-0 , $+0$)
5. Sub-Normal
6. Normal

Classification criteria

Infinity

x	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0
Sign	Exponent					Mantissa									

Sign bit: + or –

Exponent: = all 1's

Mantissa: = all 0's

Total number of possible values = $2^1 * 2^0 * 2^0 = 2$

QNaN

x	1	1	1	1	1	1	x	x	x	x	x	x	x	x	x
Sign	Exponent					Mantissa									

Sign bit: + or –

Exponent: all 1's

Mantissa: MSB = 1

Total number of possible values = $2^1 * 2^0 * 2^0 * 2^{10-1} = 2^{10} = 1024$

SNaN

x	1	1	1	1	1	0	x	x	x	x	x	x	x	x	x
Sign	Exponent					Mantissa									

Sign bit: + or –

Exponent: all 1's

Mantissa: MSB = 0 and at least one 1 (all 0's is not allowed)

Total number of possible values = $2^1 * 2^0 * 2^0 * (2^9 - 1) = 2^{10} - 2 = 1022$

The two missing values are the $+\infty$ and $-\infty$.

Zero

x	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Sign	Exponent					Mantissa									

Sign bit: + or –

Exponent: all 0's

Mantissa: all 0's

Total number of possible values = $2^1 * 2^0 * 2^0 = 2$

Subnormal

x	0	0	0	0	0	x	x	x	x	x	x	x	x	x	x
Sign	Exponent					Mantissa									

Sign bit: + or –

Exponent: all 0's

Mantissa: all 0's not allowed

Total number of possible values = $2^1 * 2^0 * (2^{10} - 1) = 2^{11} - 2 = 2046$

The two missing values are the $+0$ and -0 .

Normal

x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
Sign	Exponent					Mantissa									

Sign bit: + or –

Exponent: all 0's and all 1's are not allowed

Mantissa: all 0's not allowed

Total number of possible values = $2^1 * (2^5 - 2) * 2^{10} = 2^{16} - 2^{12} = 61,440$

Important flags

1. All bits in exponent are 0's = expZeros
2. All bits in exponent are 1's = expOnes
3. All bits in mantissa are 0's = sigZeros

Verilog Code

```
module hp_class #(
    parameter precision = 16,
    parameter exp_Length = 5,
    parameter exp_index = precision - 1 - exp_Length,
    parameter sign_length = 1
)(
    input [precision - 1 : 0]f,
    output reg SNan,    // Signalling Not a Number
    output reg QNaN,    // Quite Not a Number
    output reg Infinity,
    output reg Zero,
    output reg SubNormal,
    output reg Normal
);

    wire expZeros; // flag for exponent bit is all zeroes
    wire expOnes;  // flag for exponent bit is all ones
    wire sigZeros; // flag for significand/mantissa is all zeroes

    assign expOnes = & f[precision - 1 - sign_length : exp_index];
    // reduction operator (unary AND)
    assign expZeros = ~| f[precision - 1 - sign_length : exp_index];
    assign sigZeros = ~| f[exp_index-1 : 0]

    assign SNan      = expOnes & ~sigZeros & ~f[exp_index - 1];
    assign QNaN      = expOnes & f[exp_index - 1];
    assign Infinity  = expOnes & sigZeros;
    assign Zero      = expZeros & sigZeros;
    assign SubNormal = expZeros & ~sigZeros;
    assign Normal    = ~expOnes & ~expZeros;

endmodule
```

Verification

Type of Floating Point	Number of values
SNaN	1022
QNaN	1024
Infinity	2
Zero	2
Subnormal	2046
Normal	61440
Total	65536

We know,

$$2^{16} = 65536$$

Testbench

```
`timescale 1ns / 1ps

module tb_hp_class;
    reg [15:0]f;

    wire SNaN;
    wire QNaN;
    wire Infinity;
    wire Zero;
    wire Subnormal;
    wire Normal;

    integer i, nSNaN, nQNaN, nInfinity, nZero, nSubnormal, nNormal;

    initial begin
        assign f = 16'd0;
        nSNaN = 0;
        nQNaN = 0;
        nInfinity = 0;
        nZero = 0;
        nSubnormal = 0;
        nNormal = 0;
    end

    hp_class uut(f, SNaN, QNaN, Infinity, Zero, Subnormal, Normal);
```

```

initial begin

    for( i = 0; i < 65536; i = i + 1)begin

        #10; assign f = i;

        if(SNaN & ~QNaN & ~Infinity & ~Zero & ~Subnormal & ~Normal)begin
            nSNaN = nSNaN + 1;
        end
        else if(~SNaN & QNaN & ~Infinity & ~Zero & ~Subnormal & ~Normal)begin
            nQNaN = nQNaN + 1;
        end
        else if(~SNaN & ~QNaN & Infinity & ~Zero & ~Subnormal & ~Normal)begin
            nInfinity = nInfinity + 1;
        end
        else if(~SNaN & ~QNaN & ~Infinity & Zero & ~Subnormal & ~Normal)begin
            nZero = nZero + 1;
        end
        else if(~SNaN & ~QNaN & ~Infinity & ~Zero & Subnormal & ~Normal)begin
            nSubnormal = nSubnormal + 1;
        end
        else if(~SNaN & ~QNaN & ~Infinity & ~Zero & ~Subnormal & Normal)begin
            nNormal = nNormal + 1;
        end
        else begin
            $display("error");
        end
    end

    begin

        $display("Number of SNaN: %d", nSNaN);
        $display("Number of QNaN: %d", nQNaN);
        $display("Number of Infinity: %d", nInfinity);
        $display("Number of Zeroes: %d", nZero);
        $display("Number of Subnormal: %d", nSubnormal);
        $display("Number of Normal: %d", nNormal);

        $display("Total number of values: %d", nSNaN + nQNaN + nInfinity + nZero
+ nSubnormal + nNormal);

        #10; $stop;

    end

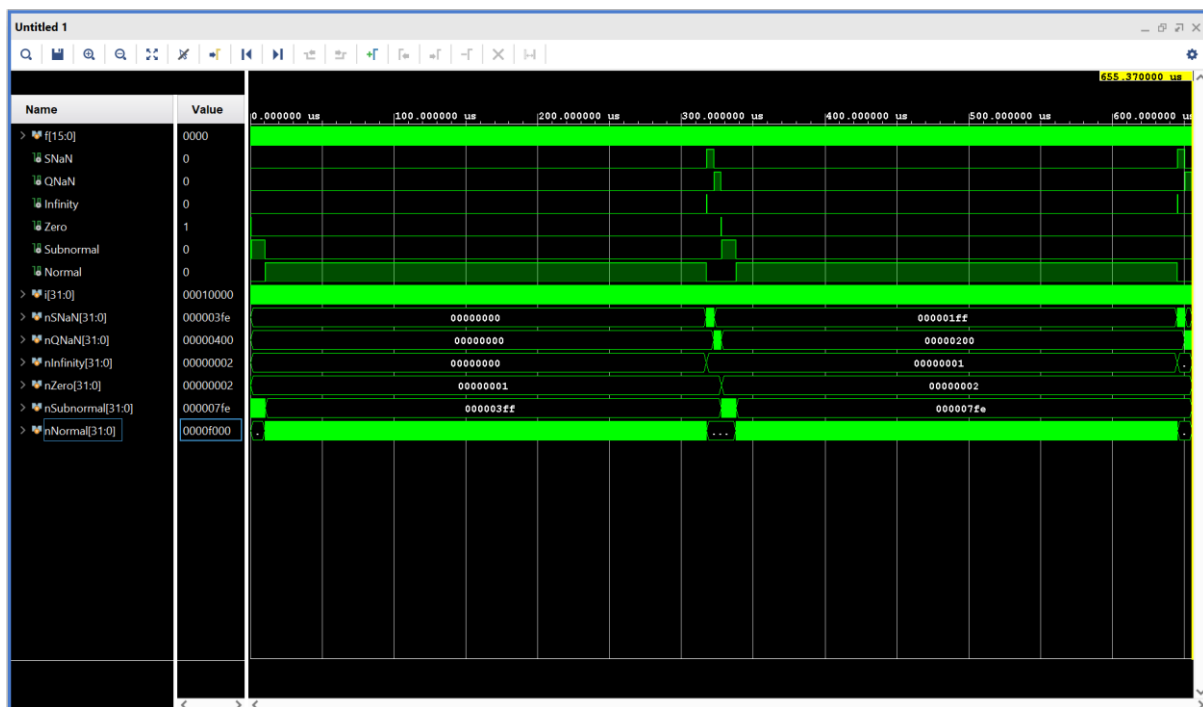
end

endmodule

```

Output

```
Number of SNaN:          1022
Number of QNaN:          1024
Number of Infinity:      2
Number of Zeroes:        2
Number of Subnormal:     2046
Number of Normal:        61440
Total number of values:   65536
$stop called at time : 655370 ns : File
"C:/Users/Amogh/Projects/TPS/Floating_Point_Classification_tb_rev_1.
v" Line 67
```



Conclusion

The given FPU classifier accurately classifies the type of floating point as described by the IEEE 754 format and passes one verification test.

Multiplication of floating point

There are 6 classifications of floating point numbers and multiplying two floating point numbers results in $6^2 = 36$ possible combinations of multiplications. By the IEEE 754 specifies how we can deal with some of the cases

	SNaN	QNaN	Infinity	Zero	Subnormal	Normal
SNaN	SNaN	SNaN	SNaN	SNaN	SNaN	SNaN
QNaN	SNaN	QNaN	QNaN	QNaN	QNaN	QNaN
Infinity	SNaN	QNaN	Infinity	QNaN	Infinity	Infinity
Zero	SNaN	QNaN	QNaN	Zero	Zero	Zero
Subnormal	SNaN	QNaN	Infinity	Zero	Zero, Subnormal	Zero, Normal, Subnormal
Normal	SNaN	QNaN	Infinity	Zero	Zero, Normal, Subnormal	Inf, Zero Normal, Subnormal

HDL code

The HDL code is split into different sections and is explained part by part. Declaring a floating-point multiplication module called "fp_mul". When this module is called, only PRECISION parameter is passed, and the rest of the parameters are calculated based on PRECISION value.

Let, x and y be the two given floating point numbers. Initializing the inputs, outputs and parameters.


```

module fp_mul #(
    parameter int PRECISION = 16,
    parameter int SIN_LEN = 1,
    parameter int EXP_LEN =
        (PRECISION == 16) ? 5:
        (PRECISION == 32) ? 8:
        (PRECISION == 64) ? 11: -1
    parameter int MAN_LEN =
        (PRECISION == 16) ? 10:
        (PRECISION == 32) ? 23:
        (PRECISION == 64) ? 52: -1

```

Defining buffers registers to hold the intermediate values during calculations. The largest value of the exponent is $(1 \ll (\text{EXP_LEN} - 1)) - 1$ and the smallest value is $-((1 \ll (\text{EXP_LEN} - 1)) - 1 - 1)$, one value is less due to zero, eg: 15 to -14 for 16-bit precision having 5 exponent bits. During multiplication the exponent bits are added. The largest resultant exponent value is

$$\begin{aligned}
 & 2 * ((1 \ll (\text{EXP_LEN} - 1)) - 1) \\
 = & 2 * (1 \ll (\text{EXP_LEN} - 1) - 2) \\
 = & (1 \ll (\text{EXP_LEN} - 1 + 1) - 2) \\
 = & (1 \ll \text{EXP_LEN}) - 2
 \end{aligned}$$

// change this to 2^{xyz} instead of right shift left shift

For 16-bit precision, the maximum value of the resultant exponent is $15 + 15 = 30$. The smallest value of exponent for normal numbers is

$$\begin{aligned}
 & 2 * (- ((1 \ll (\text{EXP_LEN} - 1)) - 1 - 1)) \\
 = & 2 * (- ((1 \ll (\text{EXP_LEN} - 1)) - 2)) \\
 = & 2 * (- ((1 \ll (\text{EXP_LEN} - 1)))) + 4 \\
 = & - ((1 \ll (\text{EXP_LEN} - 1 + 1))) + 4 \\
 = & - (1 \ll \text{EXP_LEN}) + 4
 \end{aligned}$$

For 16-bit precision, the minimum value of resultant of normal exponent is $-14 - 14 = -28$. The full range of exponent includes subnormal numbers whose exponent is of the range of $-(\text{EXP} + \text{MAN_LEN})$.

$$\begin{aligned}
 & 2 * (- ((1 \ll (\text{EXP_LEN} - 1)) - 1 - 1 + \text{MAN_LEN})) \\
 = & 2 * (- ((1 \ll (\text{EXP_LEN} - 1)) - 2 + \text{MAN_LEN})) \\
 = & 2 * (- ((1 \ll (\text{EXP_LEN} - 1)))) + 4 - (2 * \text{MAN_LEN}) \\
 = & - ((1 \ll (\text{EXP_LEN} - 1 + 1))) + 4 - (2 * \text{MAN_LEN}) \\
 = & - (1 \ll \text{EXP_LEN}) + 4 - (2 * \text{MAN_LEN})
 \end{aligned}$$

For 16 bit precision, the minimum value of resultant of exponent is $-24 - 24 = -48$.
The full range of exponent is from

$$(1 \ll \text{EXP_LEN}) - 2 \text{ to } - (1 \ll \text{EXP_LEN}) + 4 - (2 * \text{MAN_LEN}).$$

// change to 2^{xxx}

The value of exponent of subnormal numbers is very low as during multiplication we need the MSB to be '1' so we can give the result in IEEE 754 format and hence need to shift the subnormal number completely if required. From observation we can conclude the range is more than $(1 \ll (\text{EXP_LEN} + 1))$, hence let us consider the length of intermediate buffer for exponent to have an additional bit, $(1 \ll (\text{EXP_LEN} + 2))$. We observe that

$$(1 \ll (\text{EXP_LEN} + 2)) > 2 * (1 \ll (\text{EXP_LEN} + 1)) + (2 * \text{MAN_LEN}) - 6$$

The maximum length of mantissa after multiplying two mantissas of length N is $2 * N$.
Similarly, multiplying two binary number of length $\text{MAN_LEN} + 1$ is $2 * (\text{MAN_LEN} + 1)$.

```
parameter int BUF_MAN_LEN = (2 * (1 + SIN_LEN)),
parameter int BUF_EXP_LEN = EXP_LEN + 2,
parameter int BIAS = (1 << (EXP_LEN - 1)) - 1
)((
input logic [PRECISION - 1 : 0] x,
input logic [PRECISION - 1 : 0] y,

output logic signed [PRECISION - 1 : 0] p,
output logic pSNaN,
output logic pQNaN,
output logic pInfinity,
output logic pZero,
output logic pSubnormal,
output logic pNormal
);
```

Passing x and y through the fp_class module we obtain the classification of the floating-point numbers.

```
fp_class #(.PRECISION(PRECISION)) inst_x(x, a, aSign, aExp, aMan, aSNaN,
aQNaN, aInfinity, aZero, aSubnormal, aNormal);
fp_class #(.PRECISION(PRECISION)) inst_y(y, b, bSign, bExp, bMan, bSNaN,
bQNaN, bInfinity, bZero, bSubnormal, bNormal);
```

Let, a and b be the two floating points being multiplied and p be the product of the multiplication of a and b. Initializing product 'p' and evaluating sign of product

```
always@(*) begin
```

```
    p = {1'b0, {EXP_LEN{1'b1}}, 1'b0, {(MAN_LEN - 1){1'b1}}};
    pSign = aSign ^ bSign;
```

Propagating Signaling NaN and QNaN if a or b is SNaN or QNaN with priority given to a

```
if (aSNaN | bSNaN)
    begin
        p = aSNaN == 1'b1 ? a : b;
        pSNaN = 1;
    end
else if (aQNaN | bQNaN)
    begin
        p = aQNaN == 1'b1 ? a : b;
        pQNaN = 1;
    end
end
```

While multiplying infinity we come across an exception in the form of indeterminate when we perform $\infty \times 0$ which can be a SNaN storing ASCII value of '*' to indicate error due to multiplication or a QNaN. Multiplication of any number with zero results in zero.

```
else if (aInfinity | bInfinity)
    begin
        pQNaN = aZero | bZero;
        pInfinity = ~pQNaN;
        p = {pSign, {EXP_LEN{1'b1}}, pQNaN, {(MAN_LEN - 1){1'b0}}};
    end
else if (aZero | bZero) begin
    pZero = 1'b1;
    p = {pSign, {EXP_LEN{1'b0}}, {(MAN_LEN){1'b0}}};
end
end
```

Multiplication of normal * normal, normal * subnormal and subnormal * subnormal are to be handled. The bit map stored in the mantissa of a normal number only has the significant bits after the floating point and the leading '1' is assumed. The output fMan from fp_class module for a normal number add an extra MSB bit with '1' and returns for further computation. In case of a subnormal number the bit map is left shifted till the leading '1' is in the MSB position plus one extra left shift to bring it to MAN_LEN index in case it is being multiplied with a normal number.

The largest unbiased exponent of a normal number is

- $(2^{(\text{EXP_LEN} - 1)} - 1)$

The smallest unbiased exponent of a normal number is

$$-(2^{(\text{EXP_LEN} - 1)} - 2)$$

For 16-bit precision this gives

Maximum normal exponent = +15

Minimum normal exponent = -14

During multiplication, exponents are added. Therefore:

$$\begin{aligned} \text{Maximum resultant normal exponent} \\ = 2 \times (2^{(\text{EXP_LEN} - 1)} - 1) \end{aligned}$$

For 16-bit precision:

$$= 15 + 15 = 30$$

$$\begin{aligned} \text{Minimum resultant normal exponent} \\ = 2 \times (-(2^{(\text{EXP_LEN} - 1)} - 2)) \end{aligned}$$

For 16-bit precision:

$$= -14 - 14 = -28$$

The full exponent range must also include subnormals. For subnormal handling, the effective exponent can extend down to approximately

$$-(2^{(\text{EXP_LEN} - 1)} - 2 + \text{MAN_LEN})$$

After multiplication this becomes approximately

$$-2 \times (2^{(\text{EXP_LEN} - 1)} - 2 + \text{MAN_LEN})$$

Hence the intermediate exponent range spans approximately from

$$2 \times (2^{(\text{EXP_LEN} - 1)} - 1)$$

down to

$$-2 \times (2^{(\text{EXP_LEN} - 1)} - 2 + \text{MAN_LEN})$$

To safely represent this range, two extra bits are allocated for the exponent buffer:

$$\text{BUF_EXP_LEN} = \text{EXP_LEN} + 2$$

The mantissa of each operand has (MAN_LEN + 1) bits including the hidden leading 1. Multiplying two such mantissas produces up to

$$2 \times (\text{MAN_LEN} + 1)$$

```
localparam int BUF_MAN_LEN = (2 * (1 + MAN_LEN));
localparam int BUF_EXP_LEN = EXP_LEN + 2;
localparam int BIAS = (1 << (EXP_LEN - 1)) - 1;
```

Intermediate registers are declared to store extended exponent and mantissa results.

```
logic signed [BUF_EXP_LEN - 1 : 0] pExp = '0;
logic [BUF_MAN_LEN - 1 : 0] pMan = '0;
logic [BUF_MAN_LEN - 1 : 0] tempMan = '0;
logic [EXP_LEN - 1 : 0] tempExp = '0;
logic [BUF_EXP_LEN - 1 : 0] aExp, bExp;
logic [MAN_LEN : 0] aMan, bMan;
logic pSign;
logic aSign, aSNaN, aQNaN, aInfinity, aZero, aSubnormal, aNormal;
logic bSign, bSNaN, bQNaN, bInfinity, bZero, bSubnormal, bNormal;
```

Operands are classified using the fp_class module. It extracts sign, unbiased exponent, normalized mantissa and flags indicating special categories.

```
fp_class #(.PRECISION(PRECISION)) inst_a(a, aSign, aExp, aMan, aSNaN,
aQNaN, aInfinity, aZero, aSubnormal, aNormal);
fp_class #(.PRECISION(PRECISION)) inst_b(b, bSign, bExp, bMan, bSNaN,
bQNaN, bInfinity, bZero, bSubnormal, bNormal);
```

The multiplication logic is implemented inside a combinational block. All status flags are cleared initially. The product sign is obtained by XOR of operand signs.

```
always@(*)
begin

    {pSNaN, pQNaN, pInfinity, pZero, pSubnormal, pNormal} = 6'b000000;

    pSign = aSign ^ bSign;
    p = {pSign, {EXP_LEN{1'b1}}, 1'b0, {(MAN_LEN - 1){1'b1}}};
```

Signaling NaN has highest priority, followed by quiet NaN. If either operand is SNaN, it is propagated. If not, but one operand is QNaN, it is propagated.

```
if (aSNaN || bSNaN)
begin
    p = aSNaN == 1'b1 ? a : b;
```

```

        pNaN = 1;
    end
    else if (aNaN || bNaN)
    begin
        p = aNaN == 1'b1 ? a : b;
        pNaN = 1;
    end
end

```

Infinity handling includes the indeterminate case $\infty \times 0$. If one operand is infinity and the other is zero, the result is NaN. Otherwise, the result is signed infinity.

```

    else if (aInfinity || bInfinity)
    begin
        pNaN = aZero || bZero;
        pInfinity = ~pNaN;
        p = {pSign, {EXP_LEN{1'b1}}, pNaN, {(MAN_LEN - 1){1'b0}}};
    end
end

```

If either operand is zero, the result is signed zero.

```

    else if (aZero || bZero) begin
        pZero = 1'b1;
        p = {pSign, {EXP_LEN{1'b0}}, {(MAN_LEN){1'b0}}};
    end
end

```

For normal and subnormal operands, exponent and mantissa multiplication is performed. Exponents are added, and mantissas are multiplied. We used the pre normalized intermediate values from fp_class module which provide true mantissa value with the implicit '1' in the MSB.

```

    else if ((aNormal || bNormal) || (aSubnormal || bSubnormal))
    begin

        pExp = aExp + bExp;
        pMan = aMan * bMan;
    end
end

```

Normalization is required because multiplying two normalized mantissas in the range [1,2) produces a result in the range [1,4). If the MSB of pMan is 1 at the highest position, the exponent is incremented. Otherwise, if the next bit is 1, a left shift aligns the mantissa.

```

    if (pMan[BUF_MAN_LEN - 1])
    begin
        pExp = pExp + 1;
    end
    else if (~pMan[BUF_MAN_LEN - 1] && pMan[BUF_MAN_LEN - 2])
    begin
        pMan = pMan << 1;
    end
end

```

Overflow occurs when the exponent exceeds +BIAS. The result becomes infinity.

```

if (pExp > BIAS)
begin
    pInfinity = 1'b1;
    {pSNaN, pQNaN, pZero, pSubnormal, pNormal} = 5'b00000;
    p = {pSign, {EXP_LEN{1'b1}}, {MAN_LEN{1'b0}}};
end

```

If the exponent is too small even for subnormal representation, the result underflows to zero.

```

else if (pExp < -(BIAS + MAN_LEN - 1))
begin
    pZero = 1'b1;
    {pSNaN, pQNaN, pInfinity, pSubnormal, pNormal} = 5'b00000;
    p = {pSign, {EXP_LEN{1'b0}}, {(MAN_LEN){1'b0}}};
end

```

If the exponent lies within normal range:

$$-(BIAS - 1) \leq pExp \leq BIAS$$

the number is encoded as normal. The biased exponent is formed by adding BIAS.

```

else if ((pExp <= BIAS) && (pExp >= -(BIAS - 1)))
begin
    pNormal = 1'b1;
    {pSNaN, pQNaN, pInfinity, pZero, pSubnormal} = 5'b00000;
    tempExp = pExp + BIAS;
    p = {pSign, tempExp, pMan[BUF_MAN_LEN - 2 : BUF_MAN_LEN - MAN_LEN
- 1]};
end

```

If the exponent is below normal range but still representable as subnormal:

$$-(BIAS + MAN_LEN - 1) \leq pExp < -(BIAS - 1)$$

the mantissa is right shifted appropriately and exponent field is set to zero.

```

else if ((pExp < -(BIAS - 1)) && (pExp >= -(BIAS + MAN_LEN - 1)))
begin
    pSubnormal = 1'b1;
    {pSNaN, pQNaN, pInfinity, pZero, pNormal} = 5'b00000;

```



```

        tempMan = pMan >> -(pExp + BIAS - 1);
        p = {pSign, {EXP_LEN{1'b0}}, tempMan[BUF_MAN_LEN - 1 : BUF_MAN_LEN
- MAN_LEN]};
    end
end
endmodule

```

the module performs IEEE-754 compliant floating-point multiplication with complete handling of special cases, normalization, overflow, underflow and subnormal results, while using extended intermediate buffers to safely accommodate exponent and mantissa growth during multiplication.

```

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

```

Addition

```

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

```

```

module fp_add #(
    parameter int PRECISION = 16
)(
    input logic [PRECISION - 1 : 0] a,
    input logic [PRECISION - 1 : 0] b,

    output logic [PRECISION - 1 : 0] sum,
    output logic sNaN,
    output logic sQNaN,
    output logic sInfinity,
    output logic sZero,
    output logic sSubnormal,
    output logic sNormal
    // output logic ///sException,
    //output logic //sRoundOff
);

    localparam int SIN_LEN = 1;

```

```

    localparam int EXP_LEN = (PRECISION == 16) ? 5: (PRECISION == 32) ? 8:
(PRECISION == 64) ? 11: 0;
    localparam int MAN_LEN = (PRECISION == 16) ? 10: (PRECISION == 32) ? 23:
(PRECISION == 64) ? 52: 0;
    localparam int BUF_MAN_LEN = (1 << EXP_LEN) + MAN_LEN; // shouldnt ther be
a +1
    localparam int BUF_EXP_LEN = EXP_LEN + 2;
    localparam int BIAS = (1 << (EXP_LEN - 1)) - 1;

    logic sSign;
    logic signed [BUF_EXP_LEN - 1 : 0] sExp;
    logic unsigned [BUF_MAN_LEN : 0] sMan;
    logic unsigned [BUF_MAN_LEN - 1 : 0] aTempMan = '0;
    logic unsigned [BUF_MAN_LEN - 1 : 0] bTempMan = '0;

    logic [EXP_LEN - 1 : 0] sTempExp;
    logic [MAN_LEN: 0] sTempMan;

    logic unsigned [BUF_MAN_LEN : 0] mask = '1;

    logic signed [BUF_EXP_LEN - 1 : 0] aExp, bExp;
    logic unsigned [MAN_LEN : 0] aMan, bMan;
    logic aSign, aSNaN, aQNaN, aInfinity, aZero, aSubnormal, aNormal;
    logic bSign, bSNaN, bQNaN, bInfinity, bZero, bSubnormal, bNormal;

    fp_class #(.PRECISION(PRECISION)) inst_a(a, aSign, aExp, aMan, aSNaN,
aQNaN, aInfinity, aZero, aSubnormal, aNormal);
    fp_class #(.PRECISION(PRECISION)) inst_b(b, bSign, bExp, bMan, bSNaN,
bQNaN, bInfinity, bZero, bSubnormal, bNormal);
    fp_roundoff #()( );

    always @(*)
    begin

        //sException = 1'b0;
        //sRoundOff = 1'b0;

        {sSNaN, sQNaN, sInfinity, sZero, sSubnormal, sNormal} = 6'b000000;

        aTempMan [BUF_MAN_LEN - 1 : BUF_MAN_LEN - MAN_LEN - 1] = aMan;
        bTempMan [BUF_MAN_LEN - 1 : BUF_MAN_LEN - MAN_LEN - 1] = bMan;
        aTempMan [BUF_MAN_LEN - MAN_LEN - 1 - 1 : 0] = '0;
        bTempMan [BUF_MAN_LEN - MAN_LEN - 1 - 1 : 0] = '0;

        if (aSNaN || bSNaN)
        begin
            sum = aSNaN == 1'b1 ? a : b;
            sSNaN = 1;

```

```

        {sQNaN, sInfinity, sZero, sSubnormal, sNormal} = 5'b00000;
    end
    else if (aQNaN || bQNaN)
    begin
        sum = aQNaN == 1'b1 ? a : b;
        sQNaN = 1;
        {sSNaN, sInfinity, sZero, sSubnormal, sNormal} = 5'b00000;
    end
    else if (aInfinity || bInfinity)
    begin
        if((aInfinity && bInfinity) && (aSign ^ bSign))
        begin
            sQNaN = 1'b1;
            {sSNaN, sInfinity, sZero, sSubnormal, sNormal} = 5'b00000;
            sum = {1'b1, {EXP_LEN{1'b1}}, sQNaN, {(MAN_LEN - 1){1'b0}}};
            //sException = 1'b1;
        end
        else
        begin
            sInfinity = 1'b1;
            {sSNaN, sQNaN, sZero, sSubnormal, sNormal} = 5'b00000;
            sum = {aSign, {EXP_LEN{1'b1}}, {(MAN_LEN){1'b0}}};
        end
    end
    else if (aZero || bZero)
    begin
        if (aZero && bZero)
        begin
            sZero = 1'b1;
            {sSNaN, sQNaN, sInfinity, sSubnormal, sNormal} = 5'b00000;
            sum = {aSign & bSign, {EXP_LEN{1'b0}}, {(MAN_LEN){1'b0}}};
        end
        else
        begin
            sum = (bZero == 1'b1) ? a : b;
            {sZero, sSubnormal, sNormal} = (bZero == 1'b1) ? {aZero,
aSubnormal, aNormal} : {bZero, bSubnormal, bNormal};
        end
    end
    else if(aSign ^ bSign == 1 && aExp == bExp && aMan == bMan)
    begin
        sZero = 1'b1;
        sum = {aSign & bSign, {EXP_LEN{1'b0}}, {(MAN_LEN){1'b0}}};
    end
    else if ((aSubnormal || bSubnormal) || (aNormal || bNormal))
    begin
        if(aExp > bExp || (aExp == bExp && aMan > bMan) || a == b)
        begin

```

```

        bTempMan = bTempMan >> (aExp - bExp);
        sSign = aSign;
        sExp = aExp;
        sMan = '0';
        sMan [BUF_MAN_LEN - 1: BUF_MAN_LEN - MAN_LEN - 1] = aMan;

        if (aSign ^ bSign == 1)
            begin
                bTempMan = (~ bTempMan) + 1;
                sMan = sMan + bTempMan;
                sMan = sMan << 1;
            end
        else if(aSign ^ bSign == 0)
            begin
                sMan = sMan + bTempMan;
                sExp = sMan[BUF_MAN_LEN] ? sExp + 1 : sExp;
                sMan = (sMan[BUF_MAN_LEN] == 1'b0) ? sMan << 1 : sMan;
            end
        end
    else if (aExp < bExp || (aExp == bExp && aMan < bMan))
        begin
            aTempMan = aTempMan >> (bExp - aExp);
            sSign = bSign;
            sExp = bExp;
            sMan = '0';
            sMan [BUF_MAN_LEN - 1: BUF_MAN_LEN - MAN_LEN - 1] = bMan;

            if (aSign ^ bSign == 1)
                begin
                    aTempMan = (~ aTempMan) + 1;
                    sMan = aTempMan + bTempMan;
                    sMan = sMan << 1;
                end
            else if(aSign ^ bSign == 0)
                begin
                    sMan = aTempMan + bTempMan;
                    sExp = sMan[BUF_MAN_LEN] ? sExp + 1 : sExp;
                    sMan = (sMan[BUF_MAN_LEN] == 1'b0) ? sMan << 1 : sMan;
                end
            end
        end

    if(sMan != 0)
        begin
            for (integer j = 1 << ($clog2(BUF_MAN_LEN) - 1); j > 0; j = j
>> 1)
                begin
                    if ((sMan & (mask << (BUF_MAN_LEN + 1 - j))) == 0)
                        begin

```

```

        sMan = sMan << j;
        sExp = sExp - j;

    end
end
end

sTempExp = sExp + BIAS;

if (sExp > BIAS) ////// change all the below to fExp
begin
    sInfinity = 1'b1;
    {sSNaN, sQNaN, sZero, sSubnormal, sNormal} = 5'b00000;
    sum = {sSign, {EXP_LEN{1'b1}}, {MAN_LEN{1'b0}}};
    //sException = 1'b1;
end

else if (sExp < -(BIAS + MAN_LEN - 1))
begin
    sZero = 1'b1;
    {sSNaN, sQNaN, sInfinity, sSubnormal, sNormal} = 5'b00000;
    sum = {sSign, {EXP_LEN{1'b0}}, {(MAN_LEN){1'b0}}};
    //sException = 1'b1;
end

else if ((sExp <= BIAS) && (sExp >= -(BIAS - 1)))
begin
    sNormal = 1'b1;
    {sSNaN, sQNaN, sInfinity, sZero, sSubnormal} = 5'b00000;
    sum = {sSign, sTempExp, sMan[BUF_MAN_LEN - 1 : BUF_MAN_LEN -
MAN_LEN]};
end

else if ((sExp < -(BIAS - 1)) && (sExp >= -(BIAS + MAN_LEN - 1)))
begin
    sSubnormal = 1'b1;
    {sSNaN, sQNaN, sInfinity, sZero, sNormal} = 5'b00000;
    sTempMan = sMan >> -(sExp + BIAS - 1);
    sum = {sSign, {EXP_LEN{1'b0}}, sTempMan[MAN_LEN - 1 : 0]};
end

end
end
endmodule

```

//

Subtraction

//

```
module fp_sum #(
    parameter int PRECISION = 16
)(
    input logic [PRECISION - 1 : 0] a,
    input logic [PRECISION - 1 : 0] b,

    output logic [PRECISION - 1 : 0] sum,
    output logic sNaN,
    output logic sQNaN,
    output logic sInfinity,
    output logic sZero,
    output logic sSubnormal,
    output logic sNormal
    //output logic ///sException,
    //output logic //sRoundOff
);
    assign b[PRECISION - 1] = ~b[PRECISION - 1];
    fp_add #(.PRECISION(PRECISION)) inst_sub(a, b, sum, sNaN, sQNaN,
sInfinity, sZero, sSubnormal, sNormal /*, sException, sRoundOff*/);
endmodule
```

//
/

Roundoff

//
/

```
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
//////// work in progress
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

module fp_roundoff #()
```

```

parameter PRECISION
parameter MAN_LEN
parameter BUF_MAN_LEN
)((
    logic unsigned input [BUF_MAN_LEN : 0] fMan,
    logic signed input [BUF_EXP_LEN - 1 : 0] fExp,
    logic unsigned output [MAN_LEN - 1 : 0] tempMan,
    logic unsigned output [EXP_LEN - 1 : 0] tempExp
)

localparam int MSBIndex = BUF_MAN_LEN - MAN_LEN - 1;
localparam int EVEN_MAN_LEN = ~(MAN_LEN % 2);

logic [MAN_LEN + 1 : 0] rMan = '0;
logic roundMSB = [MSBIndex] fMan;
logic roundLSB = | fMan[MSBIndex - 1: 0];

always @(*)
begin
    if (roundMSB && (~EVEN_MAN_LEN || (EVEN_MAN_LEN && roundLSB)))
    begin
        rMan = fMan[BUF_MAN_LEN : BUF_MAN_LEN - MAN_LEN] + 1;
        if (rMan[MAN_LEN + 1] == 1'b1)
        begin
            tempExp = tempExp + 1;
            rMan = rMan >> 1;
            fExp = fExp + 1;
            tempMan = [MAN_LEN - 1: 0]rMan;
        end
        else if (~rMan[MAN_LEN + 2] && rMan[MAN_LEN + 1])
        begin
            tempMan = [MAN_LEN - 1 : 0] rMan
        end
    end
    else
    begin
        tempMan = [MAN_LEN - 1 : 0] rMan;
    end
end
endmodule

```